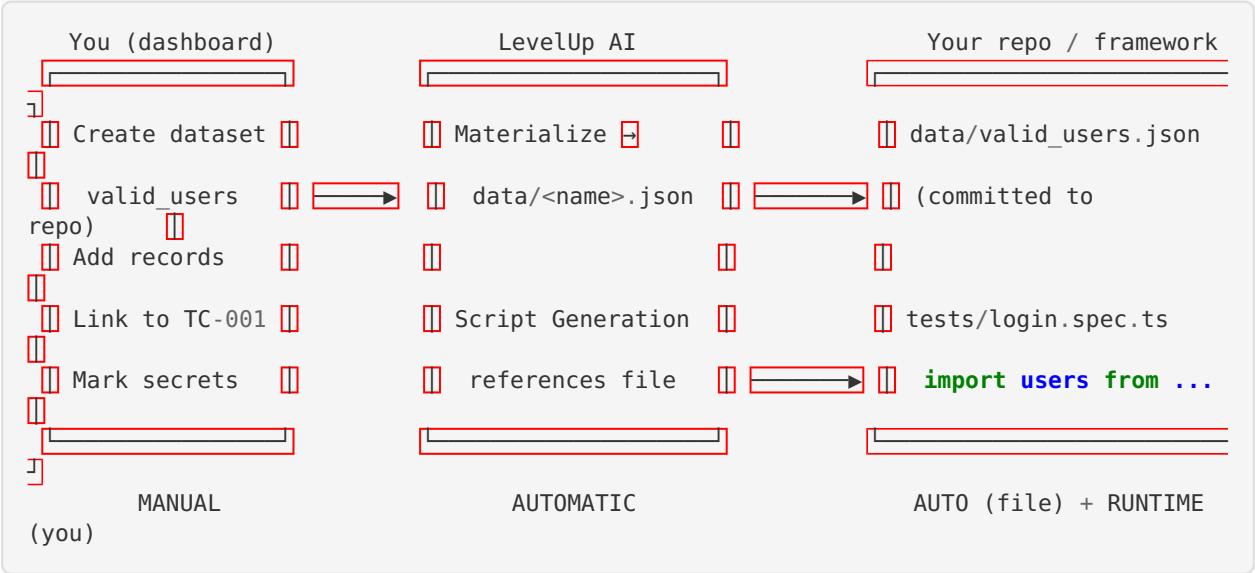


Test Data — User Guide

How to use the Test Data feature in your test cases and generated test scripts: what you set up by hand, what LevelUp AI generates for you, and what your test framework/repo needs to consume it.

1. The one-paragraph mental model

Test Data lets you store **real, reusable values** (login users, products, orders, secrets) in LevelUp — **once, per project** — so that **Script Generation references them instead of hallucinating fake values**. You manage datasets in the dashboard (**Test Data** page). LevelUp writes each dataset to a `data/<name>.json` file in your repo, links datasets to specific test cases, and then **generated Playwright scripts import those files at runtime** rather than hardcoding `user@test.com`.



2. What is automatic vs. what you set up manually

Step	Who does it	Where
Create datasets & records	You (manual)	Test Data page
Choose environment (shared/ dev/staging/prod)	You (manual)	New dataset form
Mark a value as a secret + name the env var	You (manual)	Add record form → Secret Reference
Link a dataset to a test case	You (manual)	Dataset → Linked test cases
Write <code>data/<name>.json</code> into the repo	Automatic (materialization)	<code>data/</code> folder
Discover <code>data/*.json</code> during repo scan	Automatic (Framework Auditor)	Repo intelligence
Pick the right dataset for a test case	Automatic (deterministic via linkage)	Script Generation
Generate scripts that <code>import</code> the data file	Automatic	Generated spec files
Provide the actual secret values at runtime	You (manual)	CI / hosting env vars
Read secrets in the framework (<code>process.env.X</code>)	You (manual) (one-time helper)	Your fixtures/config

Rule of thumb: Non-secret data is fully automated end-to-end. Secrets are automated up to the point of the value itself — you always supply the real secret through environment variables, never through LevelUp.

3. Concepts

Dataset

A named bucket of records, e.g. `valid_users` , `products` , `test_orders` .

- **Project-scoped** — datasets belong to one project (isolation). A dataset with no project is company-wide and shared across projects.
- **Environment-tagged** — `shared` , `dev` , `staging` , or `prod` .
- Materializes to `data/<name>.json` .

Record

A single `key` → `value` entry inside a dataset.

- `key` — a stable identifier you reference in scripts, e.g. `admin`, `user1`.
- `value` — any JSON (string, number, boolean, object, array).
- `data_type` — inferred automatically when you type the value.
- `tags` — optional labels (e.g. `login`, `smoke`) for organizing/filtering.
- `is_secret` + `secret_ref` — see [Secrets](#).

Linkage (test case ↔ dataset)

Linking a test case to a dataset makes dataset selection **deterministic**: when LevelUp generates a script for that test case, it uses **only the linked datasets**. Without a link, it falls back to all datasets in the project.

4. Step-by-step: setting up your data

4.1 Create a dataset

1. Open **Test Data** in the sidebar.
2. Click + **New Dataset**.
3. Enter a name (`valid_users`), an optional description, and pick an **Environment**.
 - The helper shows where it materializes: `data/valid_users.json`.
4. **Create dataset** — you land on the dataset detail view.

Environment fallback: resolution prefers the requested environment and falls back to `shared` when a value isn't found there. So put environment-agnostic data in `shared`, and override only what differs in `dev` / `staging` / `prod`.

4.2 Add records

1. In the dataset, click + **Add record**.
2. Enter a **Key** (`admin`).
3. Choose **Value Type**:
 - **Plain Value** — type the value. JSON is parsed automatically:
 - `user@example.com` → string
 - `42` → number, `true` → boolean
 - `{"role":"admin","email":"a@test.com"}` → object
 - **Secret Reference** — see [Secrets](#).
4. Optional **Tags** (`login`, `smoke`).
5. **Add record**.

A typical `valid_users` dataset:

Key	Value
admin	{ "email": "admin@test.com", "password": "secret_ref:ADMIN_PASSWORD" }
user1	{ "email": "user1@test.com", "password": "secret_ref:USER1_PASSWORD" }

4.3 Link the dataset to your test cases

1. In the dataset detail, find **Linked test cases**.
2. Click + **Link test case**, search, and select (e.g. `TC-001 Login`).
3. The dataset's **Used by** count updates.

Now when LevelUp generates the script for `TC-001`, it will reference **only** `valid_users` — not guess from every dataset in the project.

5. How the data reaches your scripts

5.1 The materialized file

Each active dataset becomes a JSON file in your repo's `data/` folder. The shape is an **array of records**:

```
// data/valid_users.json
[
  {
    "key": "admin",
    "value": { "email": "admin@test.com", "password": "secret_ref:ADMIN_PASSWORD" },
    "tags": ["login"]
  },
  {
    "key": "user1",
    "value": { "email": "user1@test.com", "password": "secret_ref:USER1_PASSWORD" },
    "tags": ["login"]
  }
]
```

Important: the materialized file contains the **placeholder** for secrets (`secret_ref:ADMIN_PASSWORD`), **never the real secret**. The real value is read from an environment variable at runtime (next section).

5.2 What generated scripts look like

Script Generation injects a token-safe summary of the linked datasets (name, environment, record count, sample keys — **never the raw values**) into the prompt, and instructs the model to import the file and look up records by key:

```
import { test, expect } from '@playwright/test';
import users from '../data/valid_users.json';

test.describe('Login', () => {
  test('valid admin can log in', async ({ page }) => {
    // @tc:TC1
    const admin = users.find((r) => r.key === 'admin')?.value;

    await page.goto(process.env.BASE_URL!);
    await page.getByTestId('email').fill(admin.email);
    // secret resolved from env (see §6)
    await page.getByTestId('password').fill(process.env.ADMIN_PASSWORD!);
    await page.getByRole('button', { name: 'Login' }).click();

    await expect(page.getByTestId('dashboard')).toBeVisible();
  });
});
```

The generation engine **never inlines dataset contents** — it always reads them from the file at runtime. This keeps prompts small and keeps a single source of truth.

6. Secrets (never store plaintext)

LevelUp **never stores secret values**. When you mark a record value as a **Secret Reference**, you store only the **name of an environment variable** (e.g. `ADMIN_PASSWORD`). At runtime the real value is read from `process.env`.

Setup (one-time, per environment):

1. In the record form choose **Secret Reference** and enter the env var name (`ADMIN_PASSWORD`). Optionally add a non-secret placeholder object (e.g. `{ "username": "admin" }`).
2. Provide the actual value as an environment variable wherever tests run:
 - **Locally:** `.env` / shell export — `export ADMIN_PASSWORD='...'`
 - **CI:** repository/organization secrets (GitHub Actions, etc.)
 - **Hosting (Railway):** project variables.

Two ways to consume a secret in a test:

```
// (a) Directly from the environment (simplest, recommended):
await page.getByTestId('password').fill(process.env.ADMIN_PASSWORD!);

// (b) Resolved by LevelUp's API (hydrates secret_ref → _resolved):
// GET /api/test-data/resolve?name=valid_users&environment=prod
// returns records with secrets injected from process.env on the server.
```

The resolver merges the env value into the record: for object values it adds a `_resolved` field; for scalar values it replaces the value. If the env var is missing, it logs a warning and leaves the placeholder — your test should fail loudly rather than run with a fake secret.

7. What your framework / repo needs

To consume Test Data, a repo needs only standard conventions:

1. **A `data/` folder at the repo root.** Created automatically when datasets materialize. Commit the `data/*.json` files so they're versioned and reviewable.
2. **JSON imports enabled** (Playwright + TypeScript): set `"resolveJsonModule": true` in `tsconfig.json` (already on in most setups).
3. **A base URL via env**, e.g. `process.env.BASE_URL`, used in `beforeEach`.
4. **Environment variables for every secret** referenced by your datasets (`ADMIN_PASSWORD`, ...) wherever tests run.
5. (Optional but recommended) **A tiny data helper / fixture** so tests don't repeat the `find(...)` lookup:

```
// tests/fixtures/data.ts
export function record<T = any>(dataset: { key: string; value: T }[], key: string): T {
  const hit = dataset.find((r) => r.key === key);
  if (!hit) throw new Error(`Test data record "${key}" not found`);
  return hit.value;
}

// usage:
import users from '../data/valid_users.json';
import { record } from './fixtures/data';
const admin = record(users, 'admin');
```

Nothing else is required — no SDK, no LevelUp runtime dependency in your test repo. The data files are plain JSON.

8. Environments & resolution order

When a dataset/value is resolved for environment `E`:

1. Look for the dataset in environment `E` (e.g. `prod`).
2. If not found, fall back to `shared`.
3. Resolve any `secret_ref` from `process.env`.

Practical pattern

- `shared` → values identical across environments (test user emails, product SKUs).
- `dev` / `staging` / `prod` → only the values that differ (URLs, prod-only accounts).

The dashboard **environment selector** at the top filters which datasets you see and sets the environment a new dataset is created in.

9. End-to-end example

Goal: automate `TC-001 Login` with a real admin user, secret password.

1. **Create dataset** `valid_users` (environment `shared`).

2. **Add record** `admin` → Plain Value
`{ "email": "admin@test.com" } .`
 3. **Add record** `admin_password` → Secret Reference → env var `ADMIN_PASSWORD` .
(or keep the password inside the `admin` object as a `secret_ref: placeholder`).
 4. **Link** `valid_users` to `TC-001` .
 5. **Set the env var** in CI: `ADMIN_PASSWORD=.....` .
 6. **Generate scripts** for the requirement → LevelUp emits
`tests/login.spec.ts` that imports `data/valid_users.json` and reads the secret
from `process.env.ADMIN_PASSWORD` .
 7. **Run** `npx playwright test` — the test uses real data + the real secret, with
no hardcoded values anywhere.
-

10. FAQ / troubleshooting

Q: I created a dataset but `data/<name>.json` isn't in my repo.

Materialization writes the file when datasets are created/updated **and** committed into the working repo. If you only created the dataset in the dashboard, make sure the materialization step has run for that repo/environment (see Known gap below), then commit the `data/` files.

Q: My generated test still uses a fake value.

- Confirm the dataset is **linked** to that test case (deterministic selection).
- Confirm the `data/<name>.json` file exists in the repo at generation time — the engine references the **file**, so it must be present/committed.

Q: Where do secret values live?

Only in environment variables at runtime. LevelUp stores the env var **name**, never the value.

Q: Can two projects share data?

Create the dataset as **company-wide** (no project) — it's visible to all projects. Project-scoped datasets stay isolated.

Q: How are duplicate names handled?

A dataset name is unique per `(company, project, environment)` . The same name can exist in different environments (that's how `shared` vs `prod` overrides work).

Known gap (transparency)

The materializer (`src/services/test-data-materializer.ts`) that writes `data/*.json` is implemented but is **not yet auto-triggered** on dataset create/update or during the generate-and-commit flow. Until that wiring lands you may need to materialize/commit the `data/` files explicitly. Script Generation already reads dataset **metadata** from the database and instructs generated scripts to import the files, so wiring materialization into the create/update + commit path closes the loop. (Tracked as a follow-up.)

Feature areas referenced: Test Data Store (datasets/records), Environment Support (shared/dev/staging/prod with fallback), Secret References (env-var resolution), Auto Discovery (Framework Auditor `data/` scan), Test Case ↔ Dataset Linkage (deterministic selection), and Script Generation integration.